CS383 · SOFTWARE ENGINEERING

Lecture 3 Process Models (I)

ملخص الميد

محارب

محتوى الملف

01 وش هو الـ Software process model؟

02 The Waterfall Model

03 Incremental Development

04 Reuse-oriented Development

05 الأنشطة الأربعة: Process activities

06 النشاط الأول: Specification

07 النشاط الثاني: Development

08 النشاط الثالث: V&V

09 النشاط الرابع: Evolution

10 Coping with change

11 كيف يجي في الميد؟

12 أخطاء شائعة

13 اختبر نفسك

01 وش هو الـ Software process model؟

الـ **Software process model** وصف مبسّط لأنشطة بناء البرنامج، من وجهة نظر معيّنة. تخيله خريطة طريق يمشي عليها الفريق.

ممكن يكون فيه: ترتيب الأنشطة، المدخلات والمخرجات لكل نشاط (**Inputs / Outputs**)، وأدوار الناس (**Roles**).

مجموعة الـ Processes	وين نشرحها
Plan-driven	هذي المحاضرة
Agile	المحاضرة الجاية (Lecture 4)

النماذج العامة الثلاثة

1. The Waterfall Model
2. Incremental Development
3. Reuse-oriented Development

هذي نماذج عامة (**Generic**)، والشركات تعدّل عليها عشان تسوي نموذجها الخاص.

The Waterfall Model 02

المراحل تمشي ورا بعض مثل الشلال. ما تبدأ مرحلة إلا إذا خلصت اللي قبلها.

1. Requirements Definition
2. System and Software Design
3. Implementation and Unit Testing
4. Integration and System Testing
5. Operation and Maintenance

Strengths	Weaknesses
Aligns to system engineering process يمشي مع طريقة هندسة الأنظمة.	**Inflexible to changing requirements** ما يتحمّل تغيّر المتطلبات.
Complete set of documentation وثائق كاملة لكل مرحلة.	**Late discovery of technical problems** المشاكل التقنية تنكشف متأخر.
	Sequential: next phase waits for previous phase كل مرحلة تنتظر اللي قبلها.

جاء في الميد

أول نقطتين من الـ Weaknesses جاءت نصاً كخيار صحيح في سؤال MCQ.

Incremental Development 03

النظام ينبني على دفعات. كل دفعة نسخة، والعميل يشوفها ويعطي رأيه. والأنشطة الثلاثة تشتغل مع بعض (**Concurrent activities**):

```
Specification   ⇄   Initial version
Development     ⇄   Intermediate versions
Validation      →   Final version
```

Strengths	Weaknesses
Effectively manages evolving requirements يتعامل زين مع المتطلبات اللي تتغير.	**Reduced visibility and control of activities** صعب تشوف التقدم وتتحكم في الأنشطة.
Identifies and resolves technical risk early يكشف المخاطر التقنية بدري.	**May lead to poor structured software** ممكن بنية البرنامج تخرب مع كثرة التعديلات.
Receive early customer feedback ملاحظات العميل توصل بدري.	

Reuse-oriented Development 04

اسمه الثاني **Component-based**. بدل ما نبني كل شي من الصفر، نركّب مكونات جاهزة.

1. Requirements Specification
2. Component Analysis
3. Requirements Modification
4. System Design with Reuse
5. Development and Integration
6. System Validation

Strengths	Weaknesses
Lowers the cost and time of development يقلل التكلفة والوقت.	**Compromises made on requirements** نتنازل عن بعض المتطلبات عشان تمشي مع المكونات الجاهزة.
Lowers technical risk يقلل المخاطر التقنية.	**Less control over software evolution** تحكمنا أقل في تطور النظام مستقبلاً.

05 الأنشطة الأربعة: Process activities

أي نموذج فيه نفس الأنشطة الأربعة الأساسية:

1. Software specification
2. Software development
3. Software verification and validation
4. Software evolution

جاء في الميد

سؤال طلب تكتب الأنشطة الأربعة بالترتيب المنطقي (درجتين).

الفرق بين النماذج في ترتيب الأنشطة: في **Waterfall** تمشي ورا بعض (Sequential)، وفي **Incremental** متداخلة وتتكرر (Interleaved).

06 النشاط الأول: Specification

اسمه الثاني **Requirement engineering**. هو فهم وتحديد الخدمات المطلوبة من النظام، والقيود على

تشغيله وتطويره.

المرحلة	وش يطلع منها
Feasibility study	Feasibility report
Requirements elicitation and analysis	System models
Requirements specification	User and system requirements
Requirements validation	Requirements document

07 النشاط الثاني: Development

ينقسم إلى جزأين: **Design** ثم **Implementation**.

Software design

وصف لبنية البرنامج اللي بينبني، مثل نماذج البيانات والواجهات بين المكونات. وهو عملية **Iterative**: نضيف تفاصيل ونرجع نعدّل أكثر من مرة.

Design inputs	Design activities	Design outputs
Platform information	Architectural design	System architecture
Requirements specification	Interface design	Database specification
Data description	Component design	Interface specification
	Database design	Component specification

Software implementation

ناخذ مخرجات الـ Design ونحولها إلى نظام يشتغل (**Executable system**).

الطريقة	كيف
Manual	المبرمج يكتب الكود بنفسه.
MDD	تحويل آلي للنماذج (مثل UML) إلى كود أو وثائق.
Agile	التصميم ممكن يكون موجود داخل الكود نفسه.

الاختبار أثناء التطوير

- الأخطاء (**Defects**) ممكن تطلع في أي مستوى: تصميم أو تنفيذ.
- الأخطاء تنتشر (**Defects cascade**): تغيير في مكوّن يسبب مشكلة في مكوّن ثاني.
- عشان كذا الاختبار لازم يكون **Iterative**.

08 النشاط الثالث: V&V

الـ **Verification and Validation** عملية نتأكد فيها أن النظام مطابق لمواصفاته.

#	مرحلة الاختبار	وش نسوي فيها
1	**Component testing**	نختبر كل مكوّن لحاله.
2	**System testing**	نركّب المكونات ونختبر النظام كامل.
3	**Acceptance testing**	نختبر ببيانات حقيقية من العميل.

الـ V-model

في الـ **Plan-driven** فيه خطط اختبار (**Test plans**) تربط كل مرحلة تطوير بالاختبار اللي يقابلها.

مرحلة التطوير	خطة الاختبار اللي تطلع منها
Requirements specification	Acceptance test plan
System specification	System integration test plan
System design	Sub-system integration test plan
Detailed design	Module and unit code and test

09 النشاط الرابع: Evolution

هو تغيير النظام بعد ما يشتغل عند العميل (**After deployment**).

أنواع الصيانة الثلاثة

النوع	معناه	النسبة
Fault repairs	تصليح الأخطاء.	17%
Environmental adaptation	تعديل بسبب تغيّر البيئة: جهاز أو نظام تشغيل جديد.	18%
Functional addition	إضافة أو تعديل وظائف بسبب تغيّر الشركة أو السوق.	65%

تكلفة تصليح الخطأ

نوع الخطأ	التكلفة	السبب
Coding error	رخيص	تعديل بسيط في الكود.
Design error	أغلى	يحتاج تصليح أكثر من مكوّن.
Requirement error	الأغلى	يحتاج إعادة تصميم النظام.

خطوات عملية الـ Evolution

```
Define system requirements  →  Assess existing systems
```

```
Propose system changes      →  Modify systems  →  New system
```

10 Coping with change

التغيير شي أكيد في أي برنامج: المتطلبات تتغير، والسوق يتغير، والتقنيات تتطور. والتغيير يزيد التكلفة لأنه يسبب إعادة شغل (**Rework**).

طريقتين لتقليل التكلفة

الطريقة	معناها
Change avoidance	نتوقع التغيير قبل ما يصير، قبل ما نحتاج إعادة شغل كبيرة.
Change tolerance	نصمم الـ process بحيث التغيير ينضاف بتكلفة قليلة.

أسلوبين عمليين

الأسلوب	وش هو	يدعم
System prototyping	نسخة تجريبية فيها جزء من الخدمات، لأخذ رأي العميل بدري.	avoidance
Incremental delivery	نسلّم المزايا على دفعات، والعميل يجرب ويعلّق.	avoidance و tolerance

11 كيف يجي في الميد؟

من هذي المحاضرة جاءت ثلاثة أسئلة:

1 MCQ الميد الأول

A key weakness of the Waterfall model in the slides is:

الجواب:

Inflexible to changing requirements and late problem discovery

2 **الميد الأول** MCQ

To cope with change, there are two approaches:

الجواب:

Prototyping & incremental delivery

3 **الميد الثاني** WRITTEN · 2 MARKS

What are the 4 main software engineering activities, in logical order?

الجواب:

Specification → Development → Verification and validation → Evolution

وسؤال اختيار النموذج المناسب للمشروع (4 درجات في كل ميد) يحتاج صفات النماذج من هذي المحاضرة ومن Lecture 4.

12 أخطاء شائعة

الخطأ	الصح
تقول الـ Waterfall وثائقه قليلة	بالعكس، وثائقه كاملة. هذي من نقاط قوته.
تخلط بين avoidance و tolerance	**Avoidance** نتوقع التغيير قبل. **Tolerance** نتحمله بتكلفة قليلة.
تقول prototyping يدعم الاثنين	**Prototyping** يدعم avoidance فقط.
تعكس ترتيب الاختبارات	Component ثم System ثم Acceptance.
تقول الـ Coding error أغلى خطأ	الأغلى **Requirement error**.

13 اختبر نفسك

1. أي نموذج من صفاته: **Receive early customer feedback** ؟
2. أي نموذج من صفاته: **Compromises made on requirements** ؟
3. وش الاختبار اللي يستخدم بيانات حقيقية من العميل؟
4. وش أغلى نوع خطأ عند التصليح؟ وليش؟
5. وش الأسلوب اللي يدعم avoidance و tolerance مع بعض؟

الحلول

#	الجواب	السبب
1	**Incremental development**	العميل يشوف كل نسخة ويعلّق عليها.
2	**Reuse-oriented**	نتنازل عشان تمشي مع المكونات الجاهزة.
3	**Acceptance testing**	آخر مرحلة من مراحل الاختبار.
4	**Requirement error**	يحتاج إعادة تصميم النظام.
5	**Incremental delivery**	الـ Prototyping يدعم avoidance فقط.